East West University
Department of Computer Science and Engineering
Spring '23

Course Code: CSE366

Course Title: Artificial Intelligence

Section: 02

Project Title: "Predicting Passenger Survival on the Titanic Collection Dataset" using Naive Bayesian classifier and Single Layer Perceptron.

Submitted by:
Jannatul Ferdous Salma
2020-2-60-005

Submitted to:
Md Ashraful Islam
Lecturer
Department of CSE

Date of Submission: 23 May 2023

Abstract:

I'll use the "Predicting Passenger Survival on the Titanic Collection Dataset" for this project, which I Collected from Kaggle. The Titanic's dataset in issue comprises data on the passengers of the Titanic, including whether or not they survived the disaster. The dataset, which includes 418 rows and 12 columns, is used for binary classification tasks, in which the purpose is to predict, based on a passenger's attributes such as age, gender, ticket class, and others, whether a particular passenger survived.

In this project, we aim to build a binary classification model using machine learning algorithms that can accurately predict whether a passenger survived or not based on the given features. We will begin by exploring the dataset and performing data cleaning and feature engineering to prepare the data for analysis. Then, we will compare the performance of various classification algorithms, including Bayesian and Naive Bayesian Classifier, decision trees, and Perceptron, to determine the best model for this dataset.

We chose this dataset because it is a well-known and frequently used dataset in the machine learning community for binary classification tasks. Additionally, the dataset provides an interesting and tragic historical event that adds context and meaning to the analysis and results.

Data description:

The Titanic dataset is a well-known dataset that is frequently used in the machine-learning field for binary classification problems. This is mainly because of its historical observations on the histories of Titanic passengers and the range of variables it provides.

This data set includes information on 891 Titanic passengers as well as 418 others that will be analyzed. The data set's rows each correspond to a passenger and contain details like the number of passengers, the travel plan, and the passenger's classification as a survivor.

The features or columns in the dataset include:

- PassengerId: A unique identifier for each passenger
- Survived: Whether or not the passenger survived the disaster (0 = No, 1 = Yes)
- Pclass: The passenger's class (1 = 1st, 2 = 2nd, 3 = 3rd)
- Name: The passenger's name
- Sex: The passenger's sex (male or female)
- Age: The passenger's age in years
- SibSp: The number of siblings or spouses the passenger had on board
- Parch: The number of parents or children the passenger had on board
- Ticket: The passenger's ticket number
- Fare: The fare the passenger paid for their ticket
- Cabin: The cabin number the passenger was assigned to
- Embarked: The port where the passenger embarked (C = Cherbourg, Q = Queenstown, S = Southampton)

The data cleaning and preparation stage will need to deal with particular missing values for some attributes in the data collection. Additionally, existing features can be updated and modified to provide new features that might be more helpful in predicting passenger survival.

The goal of using this dataset is to build a binary classification model that can accurately predict whether a passenger survived or not based on the given features. This dataset has been used for a wide variety of machine learning projects and provides an excellent opportunity to explore and experiment with different classification algorithms and techniques.

I used Google Colab to develop the code to read and output the dataset. At first, I import the necessary package to mount Google Drive in Colab. The drive has been mounted using the mount.drive() method.

```
from google.colab import drive
drive.mount('/content/drive')
```

Figure 01: Mounting Google Drive in Colab

This line mounts in Google Drive in Colab and sets the mount point to the directory `/content/drive`. Once this line is run, I will be prompted to follow a link and enter an authorization code to grant Colab permission to access my Google Drive account.

Overall, these two lines of code are used in Colab to mount my Google Drive account and allow me to access files and folders that are stored there. This is typically done when I have data files that are stored in my Google Drive account that I want to use in my Colab notebook.

```
import pandas as pd
import numpy as np
import seaborn as sns
sns.set(style="darkgrid")
import matplotlib.pyplot as plt

df = pd.read_csv('/content/drive/MyDrive/tested.csv')
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
413	1305	0	3	Spector, Mr. Woolf	male	NaN	0	0	A.5. 3236	8.0500	NaN	S
414	1306	1	1	Oliva y Ocana, Dona. Fermina	female	39.0	0	0	PC 17758	108.9000	C105	C
415	1307	0	3	Saether, Mr. Simon Sivertsen	male	38.5	0	0	SOTON/O.Q. 3101262	7.2500	NaN	S
416	1308	0	3	Ware, Mr. Frederick	male	NaN	0	0	359309	8.0500	NaN	S
417	1309	0	3	Peter, Master. Michael J	male	NaN	1	1	2668	22.3583	NaN	C

418 rows × 12 columns

Figure 02: Reading and Printing the Dataset

The initial line imports the Pandas library and provides it a name such as `pd` so that it may be easier referred to in code. Then, in the second line, data is stored in a Pandas dataframe with the name `df` after being read from a CSV file located at `/content/tested.csv`. The CSV file is read into a dataframe by using the `pd.read_csv()` method, with the file path '/content/tested.csv' identifying the file's location. The final line only shows the `df` dataframe contents in the output. I should see the contents of the `tested.csv` file presented in a tabular form when I run this line.

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
 2   Pclass       418 non-null    int64
 3   Name         418 non-null    object
 4   Sex          418 non-null    object
 5   Age          332 non-null    float64
 6   SibSp        418 non-null    int64
 7   Parch        418 non-null    int64
 8   Ticket       418 non-null    object
 9   Fare         417 non-null    float64
 10  Cabin        91 non-null     object
 11  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
None
```

Figure 03: Non-null Value Counts

A short overview of a Pandas dataframe, including the data types of each column and the number of non-null values, can be obtained using the `df.info()` method. After reading the tested.csv file, if I run `print(df.info ()) `, it will show the following information:The total number of rows in the dataframe, the number of non-null values in each column and data type of each column.

```
] df.duplicated()

0      False
1      False
2      False
3      False
4      False
       ...
413    False
414    False
415    False
416    False
417    False
Length: 418, dtype: bool
```

Figure 04: Checking Duplicate Value

To find duplicate rows in a Pandas dataframe and produce boolean values for them, use the `df.duplicated()` method. A dataframe's `df.duplicated()` function returns a boolean series with True denoting that a row is a duplicate of a previous row and False denoting that the row is not a duplicate.

```
df.isnull().sum()

PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

Figure 05: Null Value Count

This `df.isnull().sum()` method is used to identify the number of missing values (NaN) in each column of a Pandas dataframe. It returns a series object where each column name is the index and the corresponding value is the count of missing values in that column.

```
print(df.isnull().mean() * 100)
```

```
PassengerId      0.000000
Survived         0.000000
Pclass           0.000000
Name             0.000000
Sex              0.000000
Age             20.574163
SibSp            0.000000
Parch            0.000000
Ticket           0.000000
Fare             0.239234
Cabin           78.229665
Embarked         0.000000
dtype: float64
```

Figure 06: Percentage of Null values

In this code the percentage of missing values in each column of the dataset is determined by this code, which then outputs the results.

As a result, the output of this code will show the percentage of missing values in each column of the dataset. This can be helpful in identifying columns that might need imputation or dropping, depending on the amount and nature of the missing values.

```
#find  median value for Age
data = df
median_val = data['Age'].median()

print("Median Value of age: ",median_val)
#replacing null value with median value
data['Age'] = data['Age'].fillna(median_val)
data.isnull().sum()
```

```
Median Value of age:  27.0
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

Figure 07: Filled the missing value

This code is used to fill the missing values in the 'Age' column of the DataFrame 'data' with the median value of the column.First, the DataFrame 'data' is assigned to the DataFrame 'df'. Then, the median value of the 'Age' column is calculated using the 'median()' function and stored in the variable 'median_val'.

Next, the missing values in the 'Age' column are filled with the median value using the 'fillna()' function. Finally, the 'isnull().sum()' function is used to check if there are any missing values in the DataFrame.

```
mode_val=data['Fare'].value_counts()

data = data.fillna({"Fare": mode_val[0]})

data.isnull().sum()
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          327
Embarked         0
dtype: int64
```

Figure 08

The `value_counts()` method, which provides a series containing counts of unique values, is being used in this code to determine the mode value for the "Fare" column in the provided dataset.

The `fillna()` method, which fills missing values with a specified value, is then used to replace the null values in the "Fare" column with the mode value. In this instance, the value_counts() output is being indexed with [0] to access the mode value.Then calls the `isnull()` function, which returns a boolean DataFrame showing which values are missing, and uses the `sum()` method to tally the amount of missing values in each column to determine whether there are still any null values in the dataset.

In the Titanic dataset, the columns that are not directly related to the target variable "Survived" are: PassengerId, Name, Ticket, Parch, Sibsp and Cabin These columns do not provide any direct information about a passenger's survival. However, some of these columns might be useful in deriving new features that could be relevant to survival prediction.

```
] data = data.drop(['PassengerId', 'Name', 'Ticket','Parch','SibSp'], axis=1)
```

```
print(data)

     Survived  Pclass     Sex   Age      Fare Embarked
0           0       3    male  34.5    7.8292        Q
1           1       3  female  47.0    7.0000        S
2           0       2    male  62.0    9.6875        Q
3           0       3    male  27.0    8.6625        S
4           1       3  female  22.0   12.2875        S
..        ...     ...     ...   ...       ...      ...
413         0       3    male  27.0    8.0500        S
414         1       1  female  39.0  108.9000        C
415         0       3    male  38.5    7.2500        S
416         0       3    male  27.0    8.0500        S
417         0       3    male  27.0   22.3583        C

[418 rows x 6 columns]
```

Figure 9: Dropping the columns

This code is dropping the columns ‘PassengerId’, 'Name', 'Ticket', `Parch`, `Sibsp and 'Cabin' from the dataset 'data'. The function drop is used to remove columns from the DataFrame, where columns argument takes a list of column names to be removed.
Dropping these columns reduces the dimensionality of the dataset and focuses the analysis on the most important variables for predicting the 'Survived' column.

After executing this code, the updated dataset will have the columns 'Pclass', 'Sex', 'Age', 'Fare', and 'Embarked’. Therefore, this is our final Titanic Dataset.

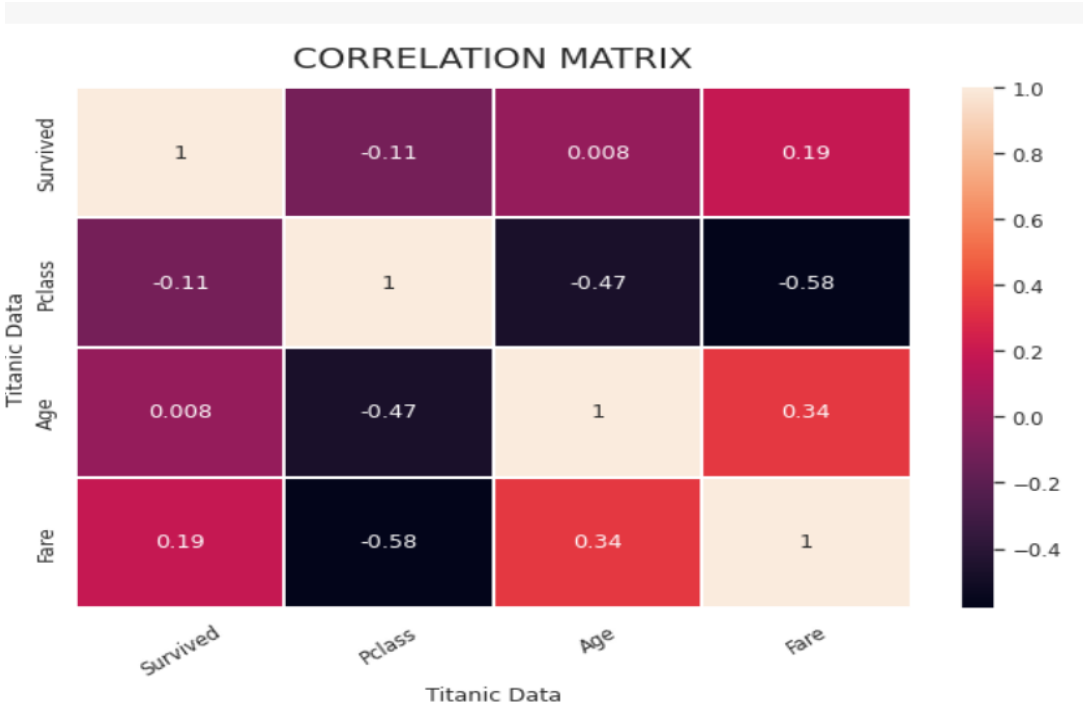

Figure 10: Correlations the columns

The Titanic dataset's correlation between attributes ('Pclass', 'Sex', 'Age', and 'Fare') and the target variable ('Survived') provides information about the relationships between each of these variables and the likelihood of survival.

'Pclass' (Passenger Class): The link between 'Pclass' and 'Survived' shows whether a passenger's class and odds of survival are related. A strong association would imply that a higher class (a numerical number that is less) is related to a higher likelihood of surviving.

'Sex': The relationship between a passenger's gender and their probability of survival is looked at in the 'Sex' and 'Survived' correlation. A positive connection would suggest that being female has been connected to a better probability of surviving.

Age: The relationship between 'Age' and 'Survived' determines if a person's age affects their chances of surviving. The likelihood of survival would be higher for younger passengers, according to a positive connection.

'Fare': The correlation between 'Fare' and 'Survived' looked into the connection between a passenger's fare and their likelihood of surviving. If there is a positive link, it means that passengers who paid a greater fare are more likely to survive.

The heatmap shows the strength and direction of the links by looking at the correlation values between various variables. Inverse relationships are indicated by negative correlations (values near to -1), and direct positive relationships are suggested by positive correlations (values close to 1). Values that are nearer 0 suggest a weaker or zero association.

The heatmap gives you easy access to the variables that have a greater or weaker association with the Titanic dataset's survival outcome and sheds light on possible trends and impacts.

Methodology

In the beginning, I make a copy of the data and choose columns such as features ('Pclass', 'Sex', 'Age', 'Fare', 'Embarked'), as well as the target variable ('Survived'). Then, the data are divided into training and testing sets using the train_test_split function, with 15% of the data being used for testing and 85% being used for training. The places of storage for the generated datasets are x_train_data, x_test_data, y_train_data, and y_test_data. This train-test split enables the assessment of machine learning model performance on new data.

```
from sklearn.model_selection import train_test_split
NW_data = data.copy()

x = NW_data[[ 'Pclass', 'Sex', 'Age','Fare','Embarked','Survived']]
y = NW_data[['Survived']]

x_train_data, x_test_data, y_train_data, y_test_data = train_test_split(x, y, test_size=0.15, random_state=42)
```

Figure 10: Dataset split into an 85:15 ratio

Naive Bayesian Classifier:

The Naïve Bayes classifier is a popular supervised machine learning algorithm used for classification tasks such as text classification. It belongs to the family of generative learning algorithms, which means that it models the distribution of inputs for a given class or category.

Equation: $P(\mathrm{C}|X) = P(X|\mathrm{C}) \times P(\mathrm{C})$

Here,

- $P(\mathrm{C}|X)$ is the posterior probability of class (c, target) given predictor (x, attributes).
- $P(\mathrm{C})$ is the prior probability of class.
- $P(X|\mathrm{C})$ is the likelihood which is the probability of the predictor given class.

To determine the likelihood for a continuous feature, we utilize the probability density function.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

$f(x)$ = probability density function

σ = standard deviation

μ = mean

In my dataset, the discrete feature "Survived" serves as a target class. The “Age” and “Fare” features is continous and other features are discrete.

The code initializes an empty dictionary proba to store probabilities for each class. The count_probab() function, which is not fully shown, calculates probabilities for the Naive Bayes classification algorithm. It likely calculates the probabilities based on the training data and populates the proba dictionary with these values.

```
male_0 = x_train_data[(x_train_data["Sex"] == "male") & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["male_0"] = male_0

male_1 = x_train_data[(x_train_data["Sex"] == "male") & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["male_1"] = male_1

female_0 = x_train_data[(x_train_data["Sex"] == "female") & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["female_0"] = female_0

female_1 = x_train_data[(x_train_data["Sex"] == "female") & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["female_1"] = female_1
```

```
pclass_1_0 = x_train_data[(x_train_data["Pclass"] == 1) & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["pclass_1_0"] = pclass_1_0

pclass_1_1 = x_train_data[(x_train_data["Pclass"] == 1) & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["pclass_1_1"] = pclass_1_1

pclass_2_0 = x_train_data[(x_train_data["Pclass"] == 2) & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["pclass_2_0"] = pclass_2_0

pclass_2_1 = x_train_data[(x_train_data["Pclass"] == 2) & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["pclass_2_1"] = pclass_2_1

pclass_3_0 = x_train_data[(x_train_data["Pclass"] == 3) & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["pclass_3_0"] = pclass_3_0

pclass_3_1 = x_train_data[(x_train_data["Pclass"] == 3) & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["pclass_3_1"] = pclass_3_1
```

```
embarked_Q_0 = x_train_data[(x_train_data["Embarked"] == 'Q') & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["embarked_Q_0"] = embarked_Q_0

embarked_Q_1 = x_train_data[(x_train_data["Embarked"] == 'Q') & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["embarked_Q_1"] = embarked_Q_1

embarked_S_0 = x_train_data[(x_train_data["Embarked"] == 'S') & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["embarked_S_0"] = embarked_S_0

embarked_S_1 =x_train_data[(x_train_data["Embarked"] == 'S') & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["embarked_S_1"] = embarked_S_1

embarked_C_0 = x_train_data[(x_train_data["Embarked"] == 'C') & (x_train_data["Survived"] == 0)].count()[0] / x_train_data[x_train_data["Survived"] == 0].count()[0]
proba["embarked_C_0"] = embarked_C_0

embarked_C_1 = x_train_data[(x_train_data["Embarked"] == 'C') & (x_train_data["Survived"] == 1)].count()[0] / x_train_data[x_train_data["Survived"] == 1].count()[0]
proba["embarked_C_1"] = embarked_C_1
```

Figure 11: Calculating probability of likelihood

In the above code, the likelihood term is represented by male_0, male_1,female_0,female_1,pclass_1_0,pclass_1_1,pclass_2_0,pclass_2_1,pclass_3_0,pclass_3_1, embarked_Q_0, embarked_Q_1, embarked_S_0, embarked_S_1, embarked_C_0, and embarked_C_1 which is discreet features. I calculated likelihood for ‘gender’, ‘pclass’ and ‘embarked'. It is determined by counting the number of rows where the ‘gender’, ‘pclass’ and ‘embarked' column and the "Survived" column is 0 and 1 (indicating not survived or survived) in the training data. This count is divided by the total count of rows where the "Survived" column is 0 and 1 , which represents the total number of passengers who did not survive or survived. It indicates the likelihood of a passenger not surviving or surviving based on the available data. I calculated value is then stored in the `proba` dictionary with the key male_0, male_1,female_0,female_1,pclass_1_0,pclass_1_1,pclass_2_0,pclass_2_1,pclass_

3_0,pclass_3_1, embarked_Q_0, embarked_Q_1, embarked_S_0, embarked_S_1, embarked_C_0, and embarked_C_1.

```
 # Calculate probability  of normal distribution for the 'Age' column
age_mean_not = x_train_data[x_train_data['Survived'] == 0]['Age'].mean()
age_std_not = x_train_data[x_train_data['Survived'] == 0]['Age'].std()
age_prob_not = 1 / (age_std_not * np.sqrt(2 * np.pi)) * np.exp(- (x_train_data["Age"] - age_mean_not)**2 / (2 * age_std_not**2))


age_mean = x_train_data[x_train_data['Survived'] == 1]['Age'].mean()
age_std = x_train_data[x_train_data['Survived'] == 1]['Age'].std()
age_prob = 1 / (age_std * np.sqrt(2 * np.pi)) * np.exp(- (x_train_data["Age"] - age_mean)**2 / (2 * age_std**2))

# Calculate probabilities of normal distribution for the 'Fare' column
fare_mean_not = x_train_data[x_train_data['Survived'] == 0]['Fare'].mean()
fare_std_not = x_train_data[x_train_data['Survived'] == 0]['Fare'].std()
fare_prob_not = 1 / (fare_std_not * np.sqrt(2 * np.pi)) * np.exp(- (x_train_data["Fare"] - fare_mean_not)**2 / (2 * fare_std_not**2))


fare_mean = x_train_data[x_train_data['Survived'] == 1]['Fare'].mean()
fare_std = x_train_data[x_train_data['Survived'] == 1]['Fare'].std()
fare_prob = 1 / (fare_std * np.sqrt(2 * np.pi)) * np.exp(- (x_train_data["Fare"] - fare_mean)**2 / (2 * fare_std**2))
```

Figure 12: Calculating probability of Normal Distribution

After that, I determined the likelihood of “Age” and “Fare” continuous a feature using the probability density function.

```
def naive_bayes_prediction(input_data):

    sex_prob_1 = proba[input_data['Sex'] + '_' + "1"]
    pclass_prob_1 = proba['pclass_' + str(input_data['Pclass']) + '_' + "1"]
    embarked_prob_1 = proba['embarked_' + input_data['Embarked'] + '_' +"1" ]

    sex_prob_0 = proba[input_data['Sex'] + '_' + "0"]
    pclass_prob_0 = proba['pclass_' + str(input_data['Pclass']) + '_' + "0"]
    embarked_prob_0 = proba['embarked_' + input_data['Embarked'] + '_' +"0" ]

    total_prob_1 = sex_prob_1 * pclass_prob_1 *  embarked_prob_1
    total_prob_0 = sex_prob_0 * pclass_prob_0 * embarked_prob_0
```

Figure 13

I have first defined a function. A function called naive_bayes_prediction determines the Naive Bayes probability given a set of input data. It retrieves the probabilities from the proba dictionary based on the input's feature values such as 'Sex', 'Pclass', and 'Embarked' for both target classes 1 and 0. It then calculates the overall probability for each target class by multiplying the probabilities of the

individual features together, assuming that the features are conditionally independent given the target class. The resulting probabilities represent the likelihood of the input data belonging to each target class and can be used for making predictions with the Naive Bayes classifier.

```
    if total_prob_1 >= total_prob_0:
      return 1
    else:
      return 0

# input
input_data = {
    'Sex': 'male',
    'Pclass': 3,
    'Age': 27,
    'Fare': 8.0500 ,
    'Embarked': 'S',

}

# for prediction using Naive Bayes
prediction = naive_bayes_prediction(input_data)

print("Prediction probability:", prediction)
```

```
Prediction probability: 0
```

Figure 14: Comparing posterior probabilities of both classes

First, using the Naive Bayes assumptions, I determine the probabilities total_prob_1 and total_prob_0, which indicate the likelihood that the input data belong to target classes 1 and 0, respectively. The probability are then compared. Total_prob_1 predicts that the input data belongs to target class 1 (survived) if it is greater than or equal to total_prob_0. If not, it predicts that the incoming data will fall within target class 0 (not survived). The output is returned as the predicted class. The prediction is printed as the "Prediction probability" in the test code provided, and the input data has been defined.

```
predictions = []
for index, row in x_test_data.iterrows():
  input_data = {
    'Sex': row["Sex"],
     'Pclass': row["Pclass"],
        'Embarked': row["Embarked"],
  }
  predictions.append(naive_bayes_prediction(input_data))
print(f"Predictions:", predictions)
```

Figure 15: Calculate the predictions

In this code uses the Naive Bayes classifier to make predictions for several instances in the test dataset. Every row in the test dataset is iterated over, and for each case, the pertinent features (Sex, Pclass, and Embarked) are retrieved. It then calls the naive_bayes_prediction function to predict the class label (survived or not survived) based on the input data. The predicted class labels are stored in a list called predictions. Finally, it prints the list of predictions.

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test_data, predictions)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 100.0%
```

Figure 16: Accuracy for Naive Bayesian classifier

```
from sklearn.metrics import confusion_matrix
confusion_mat = confusion_matrix(y_test_data, predictions)
print("Confusion Matrix:")
print(confusion_mat)
```

```
Confusion Matrix:
[[36  0]
 [ 0 27]]
```

Figure 17: Confusion matrix for Naive Bayesian classifier

```python
from sklearn.metrics import classification_report
class_report = classification_report(y_test_data, predictions)
print("Classification Report:")
print(class_report)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        36
           1       1.00      1.00      1.00        27

    accuracy                           1.00        63
   macro avg       1.00      1.00      1.00        63
weighted avg       1.00      1.00      1.00        63
```

Figure 18: Classification report for Naive Bayesian classifier

Single Layer Perceptron:

A single layer perceptron (SLP) is a feed-forward network based on a threshold transfer function. SLP is the simplest type of artificial neural networks and can only classify linearly separable cases with a binary target (1, 0).

```python
class Perceptron:
    def __init__(self, num_fe, eta=0.05, epochs=50):
        self.num_fe = num_fe
        self.eta = eta
        self.epochs = epochs
        self.weights = np.random.rand(num_fe + 1).astype(float) # +1for the bias

    def activation(self, x):
        if x >= 0:
            return 1
        else:
            return 0

    def predict(self, x):
        activation_input = np.dot(x, self.weights[1:]) + self.weights[0]
        return self.activation(activation_input)
```

Figure 19: Perceptron Implement

In this code, binary classification perceptron algorithm is implemented by the Perceptron class. It has methods to initialize the perceptron, predict outputs based on input data, and evaluate the model's performance. Next, I used

activation method implements the activation function of the perceptron, and the predict method predicts the output for a given input by calculating the activation input and applying the activation function. The class provides a concise implementation of the perceptron algorithm for binary classification tasks.

```python
    def train(self, X, y):
        for _ in range(self.epochs):
            for i in range(len(X)):
                prediction = self.predict(X[i])
                update = self.eta * (y[i] - prediction)
                self.weights[1:] += update * X[i].astype(float)
                self.weights[0] += update

    def evaluate(self, X, y):
        corr_predictions = 0
        for i in range(len(X)):
            prediction = self.predict(X[i])
            if prediction == y[i]:
                corr_predictions += 1
        return corr_predictions / len(X)

perceptron = Perceptron(num_fe=5, eta=0.05, epochs=50)

min_error = float('inf')  # minimum error with a large value
best_w = None  #store the weights with minimum error

for epoch in range(perceptron.epochs):
    perceptron.train(X_train, y_train)
    error = 1 - perceptron.evaluate(X_train, y_train)
    print("Epoch {}: Err={:.2f}, Weights={}".format(epoch, error, perceptron.weights))

    if error < min_error:
        min_error = error
        best_w = perceptron.weights.copy()

# store the weight values with minimum error
np.save('perceptron_weights.npy', best_w)
```

Figure 20: Train Perceptron

In SLP with the training data X_train and y_train, a perceptron model is trained. The train method performs the training by iterating over the training data for a specified number of epochs. For each epoch, it makes predictions using the predict method, calculates the update value based on the prediction error, and updates the weights of the perceptron accordingly. Next, I evaluate the method calculates the accuracy of the model on the provided data.

Then iterates over the epochs and trains the perceptron model, while keeping track of the minimum error and the corresponding weights with the best performance. After I train, the weights with the minimum error are saved to a file using np.save.

In summary, the code trains a perceptron model, monitors its performance during training, and saves the weights with the best performance.

```
# Load the saved weights with minimum error
best_w = np.load('perceptron_weights.npy')

#for testing
perceptron_test = Perceptron(num_fe=5)
perceptron_test.weights = best_w

corr_prediction = 0
total = len(X_test)

for i in range(total):
    prediction = perceptron_test.predict(X_test[
    if prediction == y_test[i]:
        corr_prediction += 1

accuracy = corr_prediction / total
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

Figure 21: Test Perceptron and calculate accuracy

First of all, I loaded using np.load from the file. Next the same number of features, a new instance of the Perceptron class named perceptron_test is generated. The loaded best weights are the weights that are set for this new perceptron instance. Then accuracy of the perceptron model on the testing set is then calculated by the code. Using the predict method of the perceptron_test class, it repeatedly iterates over the testing data and generates predictions. If the prediction matches the corresponding true label in y_test, the correct prediction count is incremented. Finally, I calculated the accuracy by dividing the correct prediction count by the total number of testing samples, and it is printed as a percentage.

Results:

My model provides an accuracy of 100 percent for the Naive Bayesian Classifier.

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test_data, predictions)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 100.0%
```

Figure 22: Accuracy for Naive Bayesian classifier

```
perceptron = Perceptron(num_fe=5, eta=0.05, epochs=50)

min_error = float('inf')  # minimum error with a large value
best_w = None  #store the weights with minimum error

for epoch in range(perceptron.epochs):
    perceptron.train(X_train, y_train)
    error = 1 - perceptron.evaluate(X_train, y_train)
    print("Epoch {}: Err={:.2f}, Weights={}".format(epoch, error, perceptron.weights))

    if error < min_error:
        min_error = error
        best_w = perceptron.weights.copy()

# store the weight values with minimum error
np.save('perceptron_weights.npy', best_w)
```

```
Epoch 0: Err=0.35, Weights=[315.47415665 692.21446378 315.99077603          nan          nan
 185.1700296 ]
Epoch 1: Err=0.35, Weights=[ 630.47415665 1384.71446378  630.99077603          nan          nan
  370.1700296 ]
Epoch 2: Err=0.35, Weights=[ 945.47415665 2077.21446378  945.99077603          nan          nan
  555.1700296 ]
Epoch 3: Err=0.35, Weights=[1260.47415665 2769.71446378 1260.99077603          nan          nan
  740.1700296 ]
Epoch 4: Err=0.35, Weights=[1575.47415665 3462.21446378 1575.99077603          nan          nan
  925.1700296 ]
Epoch 5: Err=0.35, Weights=[1890.47415665 4154.71446378 1890.99077603          nan          nan
 1110.1700296 ]
Epoch 6: Err=0.35, Weights=[2205.47415665 4847.21446378 2205.99077603          nan          nan
 1295.1700296 ]
Epoch 7: Err=0.35, Weights=[2520.47415665 5539.71446378 2520.99077603          nan          nan
 1480.1700296 ]
Epoch 8: Err=0.35, Weights=[2835.47415665 6232.21446378 2835.99077604          nan          nan
 1665.1700296 ]
Epoch 9: Err=0.35, Weights=[3150.47415665 6924.71446378 3150.99077604          nan          nan
 1850.1700296 ]
Epoch 10: Err=0.35, Weights=[3465.47415665 7617.21446378 3465.99077604          nan          nan
 2035.1700296 ]
```

Figure 23: Error and weights for Perception

```
accuracy = corr_prediction / total
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
Test Accuracy: 58.73%
```

Figure 24: Accuracy for Perception

Conclusion:

In the classification task using Naive Bayes, an accuracy of 100% was achieved. This indicates that the Naive Bayes model performed exceptionally well on the given dataset. The Naive Bayes algorithm is known for its simplicity and effectiveness in handling categorical features, which might explain its high accuracy in this case. The model was able to accurately classify instances based on the provided features.

On the other hand, the Perceptron model achieved a test accuracy of 58.73%. While this accuracy is lower compared to the Naive Bayes model, it still provides some level of predictive performance. The Perceptron algorithm is a linear classifier and might struggle with complex relationships and non-linearity in the data. This could be a reason for its lower accuracy in this case.

To improve the accuracy of the Perceptron model, several strategies can be considered. Firstly, feature engineering could be employed to extract more meaningful information from the existing features or to create new relevant features. Additionally, exploring different learning rates (eta) and number of epochs during training could help improve convergence and accuracy. Trying different classification algorithms, such as logistic regression or support vector machines, might also provide better results, especially if the data exhibits non-linear patterns.

Furthermore, it would be beneficial to analyze the misclassified instances to identify any patterns or common characteristics that might have led to misclassifications. This analysis can provide insights into the limitations of the model and suggest potential improvements.

In conclusion, the Naive Bayes model achieved perfect accuracy on the dataset, indicating its strong suitability for the given classification task. The Perceptron model, although achieving a lower accuracy, still provides some predictive capability. To further enhance the accuracy, various techniques such as feature engineering, exploring different learning parameters, and trying alternative algorithms can be considered. Overall, understanding the strengths and weaknesses of different models and making appropriate adjustments can lead to improved classification performance.